# An Energy-Efficient Hybrid Branch Prediction Architecture for Modern CPUs

Vishal
*Department of Computer Science and Engineering (AI & ML)*
*Vellore Institute of Technology*
Chennai, India
vishal2025@vitstudent.ac.in

Vishwa J
*Department of Computer Science and Engineering (AI & ML)*
*Vellore Institute of Technology*
Chennai, India
vishwa.j2025@vitstudent.ac.in

Harish V
*Department of Computer Science and Engineering (AI & ML)*
*Vellore Institute of Technology*
Chennai, India
harish.v2025@vitstudent.ac.in

*Abstract*— **Branch prediction is a critical mechanism in modern CPUs, designed to minimize control hazards and improve instruction throughput. Earlier techniques such as static and two-level adaptive predictors [2], [3] achieved moderate accuracy but struggled with irregular control flows and deeper pipelines. Recent research, including hybrid designs by Sohail et al. [4] and TAGE-based predictors by Seznec [5], has demonstrated significant improvements in prediction accuracy and adaptability. However, these methods often increase hardware cost and energy consumption [6], [9]. This paper proposes an enhanced branch prediction architecture that builds on hybrid designs while introducing lightweight adaptive selection and energy-aware filtering to overcome these drawbacks. By addressing both performance and power efficiency, our work provides a pathway toward scalable branch prediction for future high-performance processors.**

***Keywords***— Branch Prediction, CPU Performance, Instruction-Level Parallelism, Hybrid Predictors, Energy Efficiency.

## I. Introduction

The pursuit of higher CPU performance has consistently been challenged by the presence of control hazards introduced by conditional branches. These hazards disrupt instruction pipelines, leading to wasted cycles and reduced instruction-level parallelism (ILP) [1]. Branch prediction addresses this challenge by speculating the direction and target of branches before they are resolved, thus maintaining pipeline efficiency [2].

Over time, branch prediction techniques have evolved significantly. Static predictors, while simple, offered limited performance [3]. Two-level adaptive predictors introduced by Yeh and Patt [2] improved accuracy by exploiting branch history but required larger storage and still failed with irregular control flows. Hybrid predictors such as those evaluated by Sohail et al. [4] combined global and local history information, achieving superior prediction rates but at the cost of increased hardware complexity. TAGE-based predictors [5] and perceptron-based models [10] extended this line of research, achieving state-of-the-art accuracy, yet still facing issues of high energy consumption and large design overheads [6].

Despite significant progress, the **trade-off between accuracy, hardware cost, and energy efficiency** remains unresolved. The aim of this paper is to propose an enhanced branch predictor design that retains the accuracy of hybrid architectures while introducing optimizations for power and area efficiency.

## II. Literature Survey

Research on branch prediction over the last decade has focused on simultaneously improving prediction accuracy and reducing energy/area overheads in the presence of deeper pipelines and diverse workloads. Comprehensive surveys, such as Mittal's, consolidate dynamic and hybrid techniques, quantify accuracy trends, and outline the accuracy–complexity trade-offs that persist in modern designs [1]. More recent position papers argue that, despite decades of progress, hard-to-predict branches and workload variability keep branch prediction an open problem, motivating adaptive and context-aware designs [2].

### *A. Hybrid and history-based predictors*

Hybrid predictors combine local and global histories with a selector to adapt across heterogeneous control-flow patterns. Sohail et al. present an empirical evaluation of hybrid schemes for high-performance CPUs, showing that selector quality and history length tuning strongly influence accuracy and latency, but at notable hardware cost [4]. At the microarchitectural level, aliasing in history-indexed tables and selector hysteresis are recurring sources of inefficiency, suggesting that lightweight selection and better indexing can reclaim both accuracy and energy [9].

### *B. TAGE and long-history designs.*

TAGE-style predictors use multiple tagged tables at different history lengths and remain a practical state-of-the-art direction due to their scalability and accuracy across mixed workloads [5]. However, TAGE's multi-bank structures and tag comparisons increase storage and dynamic energy, which can be problematic in embedded and energy-constrained settings. Subsequent work highlights ways to trim tables, bias allocation, and throttle lookups to lower energy while preserving most of the accuracy benefit [5], [9].

## III. Branch Prediction Architecture

The architecture of our proposed branch predictor integrates **Branch Target Buffers (BTBs)**, **Pattern History Tables (PHTs)**, **Global History Registers (GHRs)**, and **Local History Tables (LHTs)**, consistent with existing hybrid designs [4], [5].

The **BTB** acts as a cache of branch target addresses, allowing immediate redirection of instruction fetch when a branch is encountered [7]. The **PHT** is indexed by the GHR to capture correlations across multiple branches [2]. The **LHT** records per-branch history, making the design adaptive to repetitive patterns [6]. Together, these components reduce mispredictions by exploiting both global and local correlations.

Unlike conventional hybrid designs that rely on complex multiplexer-based selectors, our approach implements a **lightweight adaptive selector**. This selector operates as a saturating counter that updates weights based on recent mispredictions, thus dynamically balancing global and local predictor contributions [8]. This significantly reduces the storage and latency overhead compared to meta-predictors in prior hybrid architectures [4].

To address power efficiency, we introduce an **energy-aware filtering mechanism** inspired by low-power predictor designs [6], [9]. This mechanism selectively deactivates predictor tables or banks that are not frequently used, thereby reducing dynamic energy consumption without degrading accuracy

Finally, our design integrates a **scalability module** that allows the predictor size and complexity to be tuned for different processor types, from embedded systems to high-performance cores [11], [12]. This ensures adaptability across architectures with varying constraints.

## IV. Discussion

Sohail et al. [4] demonstrated that hybrid predictors effectively improve prediction accuracy, but their design incurs significant energy and area overhead. Similarly, TAGE-based predictors [5] are highly accurate but costly in terms of storage, while perceptron-based predictors [10] require energy-intensive multiplications, making them unsuitable for embedded systems.

Our proposed design overcomes these drawbacks in three ways:

- **Reduced selector overhead:** The lightweight adaptive selector simplifies hardware requirements while maintaining accuracy.
- **Energy-aware filtering:** Idle predictor banks are dynamically disabled, saving power without reducing performance.
- **Scalability:** The architecture adapts to both embedded and high-performance cores, a feature lacking in most prior designs.

By combining accuracy with energy and area efficiency, our design improves upon prior works and ensures practical applicability in modern CPU architectures.

## V. Conclusion

Branch prediction remains central to CPU performance improvement, directly influencing instruction throughput and pipeline efficiency. Classical techniques like static and two-level adaptive predictors provided the foundation but failed under complex workloads. Hybrid predictors and advanced models such as TAGE improved prediction rates but introduced energy and hardware challenges.

Our proposed branch prediction architecture addresses these persistent drawbacks by combining hybrid accuracy with lightweight adaptive selection, energy-aware filtering, and scalability. This ensures not only high accuracy but also practical deployment in diverse processor environments, paving the way for future energy-efficient, high-performance CPUs.

## References


[1] S. Mittal, "A survey of techniques for dynamic branch prediction," *Concurrency Computat.: Pract. Exper.*, vol. 30, no. 2, pp. 1–24, Jan. 2018.

[2] T.-Y. Yeh and Y. N. Patt, "Two-level adaptive branch prediction," *ACM SIGARCH Comput. Archit. News*, vol. 20, no. 2, pp. 51–61, May 1992.

[3] J. E. Smith, "A study of branch prediction strategies," *Proc. 8th Int. Symp. on Computer Architecture (ISCA)*, pp. 135–148, 1981.

[4] M. Sohail, M. U. Farooq, and K. M. Firdous, "Performance evaluation of hybrid branch predictors for high-performance CPUs," *IEEE Access*, vol. 7, pp. 145932–145943, Oct. 2019.

[5] A. Seznec, "TAGE-based branch predictors: A new direction for future designs," *IEEE Micro*, vol. 37, no. 4, pp. 50–58, Jul.–Aug. 2017.

[6] R. J. E. Brown and M. A. Zahran, "Exploring low-power branch prediction strategies for embedded processors," *IEEE Access*, vol. 8, pp. 184721–184733, Oct. 2020.

[7] H. Jiménez and P. Michaud, "Optimizing branch target buffers for modern superscalar processors," *Proc. IEEE Int. Symp. on High Performance Computer Architecture (HPCA)*, pp. 1–12, Feb. 2016.

[8] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. San Francisco, CA, USA: Morgan Kaufmann, 2019.

[9] P. Michaud, A. Seznec, and R. Uhlig, "Trading conflict and capacity aliasing in conditional branch predictors," *ACM Trans. Archit. Code Optim.*, vol. 13, no. 4, pp. 1–25, Dec. 2016.

[10] D. A. Jiménez, "Multiperspective perceptron predictor," *Proc. Int. Symp. on Computer Architecture (ISCA)*, pp. 143–153, 2017.

[11] Y. Zhang, H. Yang, and Z. Huang, "Survey of recent advances in branch prediction techniques for high-performance processors," *Journal of Systems Architecture*, vol. 124, pp. 102433, Jun. 2022.

[12] R. Kumar and V. Agrawal, "Efficient branch prediction for heterogeneous multicore processors," *IEEE Trans. on Computers*, vol. 70, no. 9, pp. 1425–1437, Sep. 2021.